

Building the best free text-to-speech system

A plan for giving every book, sermon and verse we publish a natural spoken voice — in every language we serve, at a cost that stays near zero as the library grows.

Executive summary

Ochorus already speaks. Every chapter has a *Listen* mode that reads the text aloud using the voices built into the reader's own device. It costs nothing to run, works offline, and needs no audio files. For English and Spanish it is genuinely good. For Luganda it produces nothing at all, and for Swahili it produces something most listeners will switch off within a minute.

That gap is the whole problem. The languages where a spoken edition would matter most — where oral transmission is the norm, where literacy rates are lower, where a shared phone is the only device — are precisely the languages the device voices do not cover. We are strongest where we are least needed.

Closing it does not require becoming an audio company. The central move is simple: **our text does not change, so we should not be synthesising it over and over**. Unlike a chatbot, we know every sentence we will ever need to speak, in advance — which converts a per-request GPU problem into a one-time batch render plus the cost of serving static files.

There is already a plan for part of this. **docs/tts-plan.md** commits to running neural TTS *in the browser* — Kokoro or Piper via ONNX, model cached on the device — and its first phase has partly shipped. That plan is sound and this document does not replace it. But it makes one explicit concession: Swahili and Luganda fall back to device voices, “since quality neural voices for Luganda don't exist”. That concession is the gap, and §5 argues the two approaches are complementary rather than competing.

The recommendation in one paragraph

Continue the in-browser neural work for the languages it can serve. Alongside it, promote pre-rendered audio from an optional Phase 2 nicety to the primary answer for languages the device and in-browser tiers cannot reach: render the fixed corpus once with an open-licence engine, store it as Opus on the CDN, and serve it as static files. Then do the expensive thing the existing plan stops short of — build a Luganda voice, from speech recorded with native speakers. One pipeline, three consumers.

What follows

§	Section	The question it answers
2	Where we are today	What the current Listen mode does, and exactly where it fails
3	The real problem	Why low-resource languages are a different engineering problem
4	Technology landscape	Which engines are actually usable, and under which licences
5	Architecture	In-browser neural vs pre-rendered, and why both are needed
6	Getting voices	How to obtain a voice for a language that has none
7	Scripture is different	What Take Root needs that a book reader does not
8	What it costs	Compute, storage and bandwidth, with the corpus measured
9	Roadmap	How this extends the phases already in docs/tts-plan.md
10	Risks and decisions	What could go wrong, and what we need to decide

SECTION 2

Where we are today

The reader's Listen mode is implemented entirely in the browser. It holds the chapter as a list of paragraphs and speaks them one at a time through the Web Speech API, using whichever voices the operating system already provides. There is no server, no audio file and no network request involved in playback. Work is in flight: Listen has reached author biographies, and the voice and speed picker with preview has shipped.

What the current implementation gets right

- **One utterance per paragraph, not one per chapter.** This gives exact paragraph boundaries for highlight-as-you-listen, lets a speed or voice change take effect immediately by restarting only the current paragraph, and avoids the long-standing stall Chrome exhibits on long utterances.
- **Genuinely free and genuinely offline.** No per-character billing, no API key, no bandwidth. A reader on a metered connection pays nothing to listen to a book they have already downloaded.
- **Five playback speeds, a sleep timer, and Media Session integration** — so lock-screen controls and headphone buttons work, and the phone can go dark in a listener's pocket.
- **Preferences persist** (speed and chosen voice), and the code holds a reference to the active utterance to work around Chrome's habit of garbage-collecting it mid-sentence and never firing the callback that advances to the next paragraph.

Where it fails

Every one of those strengths depends on a voice existing on the listener's device. When one does not, the interface has exactly one thing to say, and it already says it: *“No voices for this language on this device.”* That string is the honest admission at the centre of this document.

Language	Device voice availability	Practical quality today
English	Universal, many voices, often neural	Good to very good
Spanish	Universal, several regional variants	Good
Portuguese	Universal, pt-BR and pt-PT	Good
Hindi	Widely present on Android and iOS	Acceptable to good
Arabic	Present, but variable; diacritics ignored	Mixed — see §7
Ukrainian	Present on recent Android and iOS; absent on older devices	Acceptable
Swahili	Sometimes present on Android; rarely elsewhere	Poor — often mispronounced or absent
Luganda	Effectively nowhere	Nothing plays at all

Availability is a property of the reader's device, not of our software, so it also varies with OS version, region and whether the user has downloaded optional voice packs. Two readers in the same city can get different results.

The other failure is silent

Even where a voice exists, it is a general-purpose one. It reads “Ps. 119:105” as letters and digits. It does not know that “Zerubbabel”, “Gethsemane” or “Chrysostom” are names. It cannot tell a scripture quotation from the author's own sentence, so a passage that the eye reads as quoted arrives in the ear as continuous prose. In devotional writing, where quotation is half the text, that flattening is a real loss of meaning.

The measured corpus

The library today holds **2,414 chapters and sermons** across seven languages, totalling roughly **6.2 million words**. At a normal reading pace that is about **685 hours** of speech — 463 of them in English. This is a large number for a person and a small number for a computer, and that asymmetry is the basis of the plan in §5.

SECTION 3

The real problem: low-resource languages

It is tempting to treat this as one problem with one solution — “add better TTS”. It is two problems, and they have almost nothing in common.

	Well-served languages	Low-resource languages
Examples in our set	en, es, pt, hi	sw, lg (and every language we add next)
What exists	Multiple good voices, already on the device	No device voice; few or no open models
The problem is	Polish — pronunciation of names, quotation prosody, reference expansion	Existence — there is nothing to polish
Cost of solving	Low; incremental improvements to what ships today	High; requires obtaining audio and training a voice
Who can judge it	Anyone	Only a native speaker
Failure if ignored	Slightly stiff listening	An entire language has no audio edition

Why the second column is hard

Modern neural TTS is trained on recorded speech paired with transcripts. For English, tens of thousands of hours are publicly available. For Luganda, the openly-licensed, studio-quality, single-speaker recordings that a high-quality voice normally requires do not meaningfully exist. Multilingual research models cover Luganda — Meta’s MMS project covers over a thousand languages — but coverage and quality are not the same thing, and as §4 shows, the licence on the broadest of those models forbids commercial use in a way that matters even for a free product.

There is also a subtler trap. A model can be intelligible and still be wrong in ways only a speaker hears: Luganda and Swahili are tonal or tone-influenced, Luganda has phonemic vowel and consonant length, and getting these wrong changes meaning rather than merely sounding foreign. A synthetic voice that mangles them is not a rough draft of a good voice. It is a liability, especially when the text is Scripture.

The governing principle, carried over from translation

We already refuse to present an unreviewed AI translation as though it were an original — it wears a badge until a native speaker approves it. Synthetic audio deserves exactly the same treatment. A generated voice in a language no reviewer has signed off on should be labelled as synthetic and unreviewed, and a listener should be able to tell, before pressing play, whether they are hearing a checked voice or a machine’s best guess.

What this implies for sequencing

The two columns want opposite orderings. The well-served languages are cheap to improve and cover most of our current readership, so they are tempting to do first. The low-resource languages are where the product is actually broken. The roadmap in §9 resolves this by making the first phase infrastructure that serves both, then spending the expensive effort on Luganda and Swahili before returning to polish.

SECTION 4

Technology landscape

“Free” has two meanings here and both matter: free for the listener at runtime, and free of licence terms that would restrict a public-domain library. We already prefer public-domain Bible texts specifically so that a quotation carries no attribution obligation; the same discipline should govern the voice that speaks them.

Engine	Licence	Strengths	Weaknesses / notes
Piper	MIT	Fast enough to run on a Raspberry Pi; dozens of languages; trains a new voice from modest data; ONNX runtime	Quality is good, not state-of-the-art. The natural default for pre-rendering and for languages we train ourselves.
Kokoro	Apache-2.0	Very high quality for its size (~82M parameters); permissive; runs on CPU	Limited language coverage; not a path to Luganda without training work.
Meta MMS-TTS	CC-BY-NC-4.0	Covers 1,100+ languages including Luganda and Swahili — by far the broadest coverage available	Non-commercial licence. Needs legal review before any use, and is a poor foundation to build a decade on.
StyleTTS2	MIT	Human-parity quality on English benchmarks; expressive prosody control	Heavier to run; training a new language is a research project, not a configuration change.
Coqui XTTS	CPML (restricted)	Voice cloning from seconds of audio; strong multilingual	Licence restricts commercial use; upstream company wound down. Avoid as a dependency.
espeak-ng	GPL-3.0	Formant synthesis; supports 100+ languages including Luganda; tiny; totally deterministic	Sounds robotic. Genuinely useful as a <i>phonemizer</i> feeding a neural model, and as a last-resort fallback.
Cloud APIs (Google, Azure, ElevenLabs)	Commercial, metered	Best quality; no infrastructure to run	Per-character billing forever; coverage of Luganda is thin to absent; creates a dependency we cannot promise to keep free.

Reading the table

- **Piper is the workhorse.** MIT, mature, trainable on the amount of audio we could realistically collect, and cheap enough that rendering the entire corpus is an afternoon rather than a budget line.
- **Kokoro is the quality upgrade** for the languages it covers, and can be swapped in per-language later without changing the architecture, because a pre-render pipeline does not care which engine produced the file.
- **MMS is the tempting shortcut and should be treated carefully.** It is the only off-the-shelf answer for Luganda, and its licence is non-commercial. Even for a free product, building the permanent audio layer of three properties on a CC-BY-NC model is a decision to make deliberately, with advice, not by default.
- **Cloud APIs are the wrong shape.** Not because they are bad — they are the best-sounding option — but because a metered per-character cost scales with listening, and the entire promise here is that listening is free.

One non-obvious component

Whatever engine we choose, we will need a **pronunciation lexicon**: an explicit mapping from the proper nouns of Scripture and church history to their phonemes, per language. This is not glamorous and it is the single highest-leverage quality improvement available, because biblical names are exactly the words a general-purpose model has never seen and exactly the words our text is full of. It is also portable — the same lexicon improves every engine we ever use.

SECTION 5

Architecture: two paths, and why we need both

The existing plan and this one agree on the goal and differ on *where synthesis happens*. That resolves cleanly once you notice the two approaches fail in opposite places.

	In-browser neural (committed plan)	Pre-rendered (this proposal)
Synthesis runs	On the listener's device, via ONNX + WebGPU/WASM	Once, on our machines, as a batch job
Delivery	An 80–120 MB model, downloaded once and cached	Opus audio per chapter, served from the CDN
Server cost	Zero	Storage and bandwidth only
Works on a cheap phone	Needs WebGPU or a fast WASM path and ~500 MB peak memory — uncertain, and exactly what the spike must answer	Yes, unconditionally; it only has to play an audio file
New language requires	A model that exists and ships to every device	A model that exists on <i>one</i> machine, once
Can it be reviewed?	No — output differs per device and per run	Yes — a file can be listened to and approved
Luganda	Cannot help; the plan falls back to device voices	Can, if we build the voice

The reconciliation

In-browser neural is a *quality* upgrade for languages that already have a model and devices that can run it. Pre-rendering is an *availability* mechanism for everything else. The committed plan already contains pre-rendering — as an optional Phase 2 item, “flagship audio for top-N books”. The argument here is only that for sw and lg it is not optional and not Phase 2: it is the only thing that works.

Three tiers, chosen per language

Tier	What it is	When it is used
0 — Device	Web Speech, unchanged	Wherever the device has a good voice; always offline
1 — In-browser neural	Kokoro or Piper via ONNX	Opt-in HD, where a model exists and the device can run it
2 — Pre-rendered	Opus per chapter + timing map	Languages tiers 0 and 1 cannot serve; offline; Take Root verses

The pipeline

Five batch steps, slotting behind the **TtsBackend** interface the committed plan already defines: pre-rendered audio is simply another backend.

Step	What happens	Why it matters
1. Segment	Split the chapter HTML into speech units, keeping paragraph boundaries and marking quotations and verse references	The paragraph is already the unit of the reader's highlight logic, so the front end barely changes
2. Normalise	Expand references and numerals to speakable words; apply the pronunciation lexicon	Where quality is won or lost, and it is language-specific
3. Synthesise	Render each unit with the language's chosen engine and voice	Batch and parallel — no user is waiting
4. Assemble	Concatenate to one Opus file per chapter plus a timing map of paragraph start times	One request per chapter; the timing map drives highlighting just as the per-paragraph callbacks do today
5. Publish	Upload behind the CDN; record voice, engine version and review state against the chapter	Provenance lets us re-render only what changed, and lets the UI badge an unreviewed voice

SECTION 6

Getting a voice where none exists

For Luganda, and for most languages we will add after it, no acceptable open voice exists. This section is about manufacturing one. It is the expensive part of the plan and the part that cannot be rushed.

What a trainable voice actually requires

Ingredient	Realistic target	Notes
Recorded speech	5–10 hours from one speaker; 1–2 hours can produce a usable voice by fine-tuning a multilingual base	Single speaker, consistent microphone, quiet room. Consistency matters more than studio equipment.
Transcripts	Exact, verified, matching the audio word for word	We have an advantage here: we can supply the script from our own public-domain corpus, so the text is already clean and already licensed.
A phonemizer	espeak-ng already supports Luganda and Swahili	Converts text to phonemes so the model learns sound, not spelling.
A speaker	A native speaker willing to be the voice of the library	This is a relationship, not a procurement. Consent must be explicit, written, and specific about permanence and scope.
A reviewer	A second native speaker to judge the output	The person who records should not be the only person who approves.

Three routes, in order of preference

- **Record our own.** Highest quality, clearest licensing, and it produces something reusable: an openly-licensed Luganda speech dataset that did not previously exist, which others can use. The cost is coordination and a few weeks of a speaker's time. Recording our own public-domain text sidesteps every rights question about the script.
- **Fine-tune an existing multilingual model** on a smaller amount of recorded audio. Faster and cheaper, but the licence of the base model propagates to the result — which is exactly the reason to avoid a non-commercial base.
- **Use existing open corpora** (Common Voice, OpenSLR and similar) where they cover the language. Cheapest, but coverage for Luganda is thin, and crowd-sourced recordings are many speakers in many rooms, which suits speech *recognition* far better than speech synthesis.

The ethical frame, stated plainly

A synthetic voice is a person's voice. Anyone who records for us should know, in their own language and in writing, what the recording will be used for, that the resulting voice can say sentences they never spoke, that it will be published freely, and how to ask for it to be withdrawn. They should be credited unless they ask not to be, and paid if they want to be paid. None of this is legally forced on us; all of it is the difference between a gift and an extraction.

The same care applies to whose voice we do *not* use. No cloning a preacher's voice from sermon recordings, however technically easy and however public-domain the text. A voice is not in the public domain because the words are.

SECTION 7

Scripture is different: what Take Root needs

A Bible API has requirements a book reader does not, and they are specific enough to design for rather than discover later.

Requirement	Why it exists	Design consequence
Verse-addressable audio	Callers ask for John 3:16–18, not for a chapter. Any range must be playable.	Render per verse and store per-verse timings; assemble ranges by concatenation or by byte-range within a chapter file. Never re-synthesise per request.
Verse numbers are optional	Spoken aloud, “sixteen” between sentences is an interruption; for study it is essential.	Store number announcements as separate, skippable units so both modes come from one render.
Absolute textual fidelity	Scripture wording must come from the trusted Bible text, never from a model's memory or paraphrase.	The normalisation step may change <i>how a word is pronounced</i> . It may never change <i>which words are spoken</i> . This should be enforced by a test, not by discipline.
A proper-noun lexicon per language	Biblical names are the highest-frequency out-of-vocabulary words in the entire text.	Build it once per language, version it, and share it with Ochorus — the same names appear in every devotional we publish.
Per-translation rendering	Each language has its own authoritative text (Reina-Valera 1858, Van Dyck, Kulish, and so on).	Audio is keyed by translation code, not by language. Two Spanish translations are two renders.
Reverence in prosody	Scripture read at the pace and pitch of a news bulletin sounds wrong to the people who care most about it.	Slower default rate, longer inter-verse pauses, and native-speaker review as an acceptance criterion rather than a nicety.

Why this belongs in Take Root rather than in each app

Ochorus quotes Scripture constantly, in every language, and already sources that wording from Take Root rather than from any model. If Take Root serves the audio for a verse as well as its text, then a quotation inside a devotional can be spoken in the same voice, from the same trusted text, as the same verse read in the Bible reader. Building the audio layer anywhere else guarantees two renderings of the same verse that drift apart.

A concrete scale check

A complete Bible read aloud is roughly 75 hours. Across the eight languages we currently target that is about 600 hours of audio — comparable to the 685 hours already implied by the Ochorus library, and equally a one-time cost. At Opus bitrates suitable for speech, a whole Bible is on the order of 1–2 GB per translation.

SECTION 8

What it costs

The numbers below use the measured corpus: 2,414 chapters and sermons, 6.2 million words, about 685 hours of speech at a normal reading pace. They are order-of-magnitude estimates intended to establish whether this is a hundred-dollar problem or a hundred-thousand-dollar one.

Line item	Estimate	Basis
One-time render of the whole library	Tens of dollars of compute	Piper synthesises far faster than real time on commodity CPU; 685 hours of audio is a batch job measured in CPU-hours, parallelised across cheap workers.
Storage	~30 GB for the library; ~1–2 GB per Bible translation	Opus at speech-appropriate bitrates is roughly 0.5–1 MB per minute of speech at generous quality, less at conservative settings.
Bandwidth	The only cost that scales with usage	A chapter is a few megabytes. Public-domain audio never expires, so edge caching is near-total and origin egress stays small.
Training one new voice	Low hundreds of dollars of GPU time, plus weeks of human coordination	The compute is not the expensive part. Finding, recording and reviewing with a speaker is.
Re-render after a text fix	Cents	Only the changed chapters re-render, because provenance is recorded per chapter.
On-demand tier	Small and bounded	Serves only text with no pre-render yet; every request it answers becomes a cached file.

The comparison that decides the architecture

A commercial TTS API priced per character would charge for the same sentence every time anyone listened to it. Our corpus is 6.2 million words; at typical commercial rates a *single* full pass over it already runs into the hundreds of dollars, and every subsequent listener pays again. The pre-render approach pays once, in the tens of dollars, and then serves bytes. Ten thousand listeners cost the same to synthesise as one.

This is the entire economic argument, and it holds regardless of which engine wins the quality comparison. It is why the recommendation is a pipeline shape rather than a model choice: the shape is what makes free sustainable, and the model inside it can be replaced whenever something better appears.

What is not in these numbers

Human review time, which is the real budget. Someone has to listen to a Luganda chapter and say whether it is right. Estimating that in dollars misses the point — it is the same native-speaker review capacity that the translation work already needs, and it is the genuine constraint on how fast any of this ships.

SECTION 9

Roadmap

This extends the phases in **docs/tts-plan.md** rather than replacing them. Phases 0 and 1 there stand as written; what follows adds a track beside them and then converges.

Track A — Continue the committed plan, unchanged

Phase 0 polish, then the spike, then in-browser neural

- Finish Phase 0: engine-interface refactor, account-synced voice prefs, sentence-level boundary highlighting, auto-advance.
- Run the spike as specified; its answer determines how much Track B has to carry.
- Ship in-browser neural where the gates pass; opt-in elsewhere.

Track B — Worth doing whatever the spike says

Small, front-end, no new infrastructure, no dependency on Track A

- Tell the reader the truth about voice availability *before* they press play, not after.
- Ship a pronunciation lexicon for biblical proper nouns; the same lexicon later feeds every engine.
- Expand references and numerals into speakable words at synthesis time, without touching the stored text.
- Instrument what people listen to, and in which languages.

Phase 2' — Build the pre-render pipeline properly

Promoting the committed plan's optional item to real infrastructure

- Stand up the five-step pipeline of §5 as a batch job over the fixture content, behind the existing TtsBackend interface.
- Render English first — we can evaluate it ourselves and it exercises every part of the pipeline.
- Extend offline download to include a book's audio.
- Record engine, voice and version per chapter so re-rendering is incremental.

Phase 3 — Close the low-resource gap

The part the committed plan explicitly sets aside; the point of this document

- Resolve the MMS licence question and choose the licence-safe engine path for Luganda and Swahili.
- Recruit and record native speakers; publish the resulting speech dataset openly if they consent.
- Train and evaluate with native reviewers, gated by the same approval discipline as translation — an unreviewed voice is badged, never presented as final.
- Render and ship the Luganda and Swahili libraries.

Phase 4 — Scripture audio and the shared service

Where the three properties converge

- Extend the pipeline to verse granularity and render the Bible translations already configured per language.
- Serve verse-range audio from Take Root so a quotation in a devotional and the same verse in the Bible reader are one recording.
- Expose the pipeline as a shared service a third consumer can adopt.

Sequencing note

Track B is worth doing whatever else happens: it improves what ships today and produces the listening data the later phases depend on. Phase 3 is the one that must not be quietly dropped once Track A proves satisfying — the committed plan sets it aside, and that is the decision this document asks to revisit.

SECTION 10

Risks, open questions, decisions needed

Risks

Risk	Consequence	Mitigation
A non-commercial model licence becomes load-bearing	The audio layer of three properties rests on terms we cannot honour	Settle the MMS licence question before phase 3, not during it. Prefer MIT or Apache-licensed engines even at some quality cost.
Synthetic Scripture that mispronounces meaning-bearing tone or length	Text that is doctrinally wrong to the ear while being correct on the page	Native review as an acceptance gate, not a follow-up. Badge unreviewed voices exactly as unreviewed translations are badged.
Storage and bandwidth grow faster than expected	A cost that was supposed to be near zero becomes a monthly line item	Conservative Opus bitrates; aggressive edge caching; render on first demand rather than everything up front if usage data justifies it.
Speaker consent handled casually	A person's voice used in ways they did not expect — a real harm and an irreversible one	Written, specific, translated consent, with a withdrawal path, before any recording session.
The pipeline becomes a second content system	Audio drifts out of step with corrected text	Provenance per chapter and re-render on text change, driven by the same correction chain that already fixes bodies at deploy.
Effort concentrates on English because it is easy to evaluate	We finish the part that was already working and never ship Luganda	Phase 3 is explicitly the point of the plan; treat it as the success criterion rather than as an extension.

Open questions

- **Every Tongue.** This document treats it as a third consumer of the same shared service, because that is the shape the architecture suggests. Its actual requirements are unknown here — nothing about it exists in the codebase — so this is an assumption, flagged as one. If it has different needs, particularly around live or user-supplied text, §5's balance between the pre-rendered and on-demand tiers is the part that would change.
- **Do we publish the datasets and voices we create?** Recommended yes, with speaker consent: an openly-licensed Luganda voice would be a genuine contribution, and it costs us nothing we are keeping.
- **Where does the batch pipeline run?** It has no latency requirement, so it can be the cheapest compute we can find rather than anything co-located with the API.
- **Human narration for the highest-value works?** A well-read chapter beats any synthetic voice. It does not scale to 2,414 chapters, but it may be right for a handful, and the same delivery pipeline serves both.

What we need to decide first

#	Decision	Blocks
1	Do we add the pre-render track alongside the committed in-browser plan, or wait for the spike result first?	Phase 2' onward
2	What licence floor do we hold engines and models to?	Phase 3 engine choice
3	Is Luganda the first low-resource target, or Swahili?	Phase 3 recruitment
4	Who owns native-speaker review capacity for audio, given the same people are needed for translation review?	Phase 3 and 4 timelines

The short version: we already have the hard part — a clean, licensed, finite corpus in eight languages, and a reader that knows how to speak it. What is missing is a voice for the people who need it most, and a pipeline that makes giving them one a fixed cost rather than a recurring one.